

““

Ford–Fulkerson and Edmonds–Karp Algorithms

Design and Analysis of Algorithms (DAA)

Name: Advait Babu K

Register Number: 24100943

““

1 Introduction

Many real-world problems can be represented using a network of connected points. Examples include transportation networks, computer networks, water pipelines, communication systems, and traffic networks.

In such problems, we may want to find the maximum amount of flow that can be sent from a starting point to a destination point. The starting point is called the source and the destination point is called the sink.

The Maximum Flow problem is one of the important problems in graph algorithms. Two well-known algorithms used to solve this problem are:

1. Ford–Fulkerson Algorithm
2. Edmonds–Karp Algorithm

Both algorithms repeatedly find an augmenting path and increase the flow until no more possible paths are available.

2 Flow Network

A flow network is a directed graph in which every edge has a capacity.

A flow network can be represented as:

[$G=(V,E)$]

where:

- V is the set of vertices.
- E is the set of directed edges.
- s is the source vertex.
- t is the sink vertex.
- $c(u,v)$ is the capacity of edge (u,v) .
- $f(u,v)$ is the current flow through edge (u,v) .

The flow must satisfy two important conditions.

2.1 Capacity Constraint

For every edge (u,v) ,

[$0 \leq f(u,v) \leq c(u,v)$]

This means that the flow through an edge cannot be greater than its capacity.

2.2 Flow Conservation

For every vertex other than the source and sink, the amount of incoming flow must be equal to the amount of outgoing flow.

[$\sum_u f(u,v) = \sum_w f(v,w)$]

Therefore, flow cannot be created or destroyed at an intermediate vertex.

3 Residual Graph

The residual graph is an important concept used by the Ford–Fulkerson algorithm.

For every edge (u, v) , the residual capacity is:

$$[c_f(u, v) = c(u, v) - f(u, v)]$$

The residual graph also contains reverse edges. These reverse edges allow the algorithm to cancel some previously assigned flow if a better path is found later.

For example, if an edge has capacity 10 and currently carries flow 6, then its remaining forward capacity is:

$$[10-6=4]$$

The corresponding reverse edge can have residual capacity 6.

This allows the algorithm to send flow backwards and modify an earlier decision.

4 Augmenting Path

An augmenting path is a path from the source s to the sink t in the residual graph.

If an augmenting path exists, additional flow can be sent through that path.

The amount of additional flow that can be sent is called the bottleneck capacity.

It is given by:

$$[\Delta = \min_{(u, v) \in P} c_f(u, v)]$$

where P is the augmenting path.

After finding the bottleneck, the flow along every edge of the path is increased by Δ .

5 Ford–Fulkerson Algorithm

The Ford–Fulkerson algorithm is a method for finding the maximum flow in a flow network.

The main idea is simple:

1. Start with zero flow.
2. Find any augmenting path from the source to the sink.
3. Find the minimum residual capacity on that path.
4. Increase the flow along the path by that amount.
5. Update the residual graph.
6. Repeat until there is no augmenting path.

When no augmenting path exists, the current flow is a maximum flow.

5.1 Ford–Fulkerson Algorithm Steps

““

Step 0: Initialize the flow on every edge to zero.

Step 0: Construct the residual graph using the capacities of the edges.

Step 0: Find any path from the source s to the sink t in the residual graph.

Step 0: Find the minimum residual capacity on this path:

$$\Delta = \min_{(u,v) \in P} c_f(u,v)$$

Step 0: Increase the flow along the path by Δ .

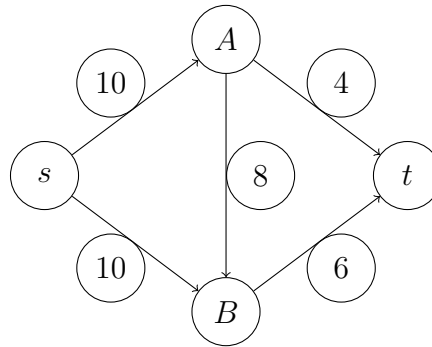
Step 0: Update the residual capacities, including the reverse edges.

Step 0: Repeat the process until there is no path from s to t in the residual graph.

Step 0: The resulting flow is the maximum flow. ““

6 Example of Ford–Fulkerson

Consider the following flow network.



Suppose the first augmenting path selected is:

[$s \rightarrow A \rightarrow t$]

The capacities on this path are:

[10, 4]

Therefore, the bottleneck capacity is:

[$\Delta = \min(10, 4) = 4$]

The flow of 4 units is sent through this path.

Next, suppose the algorithm selects:

[$s \rightarrow B \rightarrow t$]

The capacities available on this path are:

[10, 6]

Therefore:

[$\Delta = \min(10, 6) = 6$]

The flow is increased by 6 units.

The total flow is now:

[$4+6=10$]

The algorithm continues searching for augmenting paths until none are available.

7 Time Complexity of Ford–Fulkerson

The running time of Ford–Fulkerson depends on how augmenting paths are selected and on the capacities of the edges.

If all edge capacities are integers, every augmentation increases the flow by at least one unit.

Let:

[F = value of the maximum flow]

Then there can be at most F augmentations.

Finding an augmenting path using DFS or BFS takes:

[$O(E)$]

time.

Therefore, the total time complexity is:

[$O(EF)$]

for integer capacities.

7.1 Important Point

The complexity $O(EF)$ depends on the value of the maximum flow. Therefore, Ford–Fulkerson is not considered a polynomial-time algorithm when the capacities are represented in binary.

Also, with irrational capacities, Ford–Fulkerson can fail to terminate for some choices of augmenting paths.

8 Edmonds–Karp Algorithm

Edmonds–Karp is a specific implementation of the Ford–Fulkerson method.

The main difference is the way an augmenting path is selected.

Ford–Fulkerson can use any augmenting path.

Edmonds–Karp always selects the augmenting path with the smallest number of edges.

This path is found using Breadth-First Search (BFS).

Therefore:

[$\text{Edmonds–Karp} = \text{Ford–Fulkerson} + \text{BFS}$]

9 Why BFS is Used

BFS explores a graph level by level.

Therefore, when BFS reaches the sink, the path found contains the minimum possible number of edges.

This gives Edmonds–Karp a predictable polynomial running time.

The algorithm does not depend on the numerical values of the capacities for its running time.

10 Edmonds–Karp Algorithm Steps

““

Step 0: Initialize the flow on every edge to zero.

Step 0: Construct the residual graph.

Step 0: Use BFS to find the shortest augmenting path from the source s to the sink t .

Step 0: If no path is found, stop. The current flow is the maximum flow.

Step 0: Find the bottleneck capacity of the path:

$$\Delta = \min_{(u,v) \in P} c_f(u, v)$$

Step 0: Increase the flow along the selected path by Δ .

Step 0: Update the residual capacities and reverse edges.

Step 0: Repeat the BFS search until no augmenting path exists.

Step 0: Return the resulting maximum flow. ““

11 Time Complexity of Edmonds–Karp

In Edmonds–Karp, BFS is used to find every augmenting path.

The time required for one BFS is:

[$O(E)$]

A key property of Edmonds–Karp is that the number of augmentations is bounded by:

[$O(VE)$]

Therefore:

[$O(E) \times O(VE)$]

gives:

[$O(VE^2)$]

Thus, the time complexity of Edmonds–Karp is:

[$O(VE^2)$]

where:

- V is the number of vertices.
- E is the number of edges.

This is a polynomial-time complexity.

12 Correctness of the Algorithms

The correctness of both algorithms is based on the Max-Flow Min-Cut theorem.

The theorem states that the maximum possible flow from the source to the sink is equal to the minimum capacity of an s - t cut.

Theorem 1. *If there is no augmenting path from s to t in the residual graph, then the current flow is a maximum flow.*

12.1 Reason

The algorithm continuously increases the flow as long as an augmenting path exists.

When no augmenting path remains, the vertices reachable from the source in the residual graph form a cut.

All edges crossing this cut from the reachable side to the unreachable side are fully saturated.

Therefore, the capacity of this cut is equal to the value of the current flow.

By the Max-Flow Min-Cut theorem, the current flow must be a maximum flow.

13 Comparison of Ford–Fulkerson and Edmonds–Karp

Feature	Ford–Fulkerson	Edmonds–Karp
Path selection	Any augmenting path	Shortest augmenting path in terms of edges
Search method	DFS or any suitable method	BFS
Running time	$O(EF)$ for integer capacities	$O(VE^2)$
Dependence on capacities	Depends on maximum flow value	Independent of capacity values for the asymptotic bound
Polynomial time	Not guaranteed in general	Yes
Implementation	Simpler	More structured because BFS is used

14 Important Terms

““

Flow: The amount of a commodity sent through a network.

Capacity: The maximum amount of flow that an edge can carry.

Source: The vertex from which the flow originates.

Sink: The vertex where the flow terminates.

Residual Capacity: The remaining capacity available on an edge.

Residual Graph: A graph representing the remaining possible flow and reverse flow.

Augmenting Path: A path from the source to the sink in the residual graph.

Bottleneck Capacity: The smallest residual capacity on an augmenting path.

Maximum Flow: The largest possible flow from the source to the sink. ““

15 Complexity Summary

The main complexities can be summarized as follows.

Algorithm	Time Complexity
Ford–Fulkerson	$O(EF)$ for integer capacities
Edmonds–Karp	$O(VE^2)$

Here, V represents the number of vertices, E represents the number of edges, and F represents the value of the maximum flow.

16 Key Points

The following points are important to remember:

““

1. Ford–Fulkerson is a method for finding maximum flow.
2. It starts with zero flow.
3. It repeatedly finds an augmenting path.
4. The bottleneck capacity determines how much additional flow can be sent.
5. The residual graph contains both forward and reverse capacities.
6. Ford–Fulkerson can choose any augmenting path.
7. For integer capacities, its complexity is:

$$O(EF)$$

8. Edmonds–Karp is a specific implementation of Ford–Fulkerson.
9. Edmonds–Karp uses BFS to find the shortest augmenting path.
10. The time complexity of Edmonds–Karp is:

$$O(VE^2)$$

11. Edmonds–Karp always terminates in polynomial time.
12. Both algorithms are based on repeatedly increasing the flow using augmenting paths. ““

17 Conclusion

Ford–Fulkerson and Edmonds–Karp are important algorithms for solving the Maximum Flow problem.

Ford–Fulkerson provides the basic idea of repeatedly finding augmenting paths in a residual graph. Its performance depends on the choice of paths and, for integer capacities, can be expressed as $O(EF)$.

Edmonds–Karp improves the path-selection method by always using BFS to find the shortest augmenting path. This gives it a polynomial time complexity of:

[$O(VE^2)$]

The main difference to remember is that Ford–Fulkerson allows any augmenting path, whereas Edmonds–Karp specifically uses BFS.